

Practice Questions: Spring Bean Scope, Lazy/Eager Initialization & Circular Dependency

Common Assumptions

Use these assumptions for all questions unless mentioned otherwise:

1. This is a Spring Core application.
2. We are using annotation-based configuration.
3. We are NOT using Spring Boot.
4. We are NOT using `@SpringBootApplication`.
5. We are NOT using `application.properties`.
6. All classes are present inside the package: `in.strikes`
7. Ignore Spring internal logs.
8. Only consider output printed using `System.out.println()`.
9. Singleton is the default scope.
10. Singleton beans are eagerly initialized by default.
11. Prototype beans are created only when requested.

Common configuration class:

```
package in.strikes;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("in.strikes")
public class AppConfig {
}
```

For Correct answers refer Answer key last.

Questions

Question 1: Basic Eager Initialization

Code

```
package in.strikes;

import org.springframework.stereotype.Component;

@Component
public class PaymentGateway {

    public PaymentGateway() {
        System.out.println("PaymentGateway created");
    }
}
```

```
package in.strikes;

import org.springframework.stereotype.Component;

@Component
public class PaymentService {

    public PaymentService(PaymentGateway paymentGateway) {
        System.out.println("PaymentService created");
    }
}
```

```

package in.strikes;

import org.springframework.stereotype.Component

@Component
public class OrderService {

    public OrderService(PaymentService paymentService) {
        System.out.println("OrderService created");
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppCon
fig.class);

        System.out.println("Application started");
    }
}

```

Question

What will be the output?

Question 2: Lazy Bean Not Requested

Code

```
package in.strikes;

import org.springframework.context.annotation.*;
import org.springframework.stereotype.Component;

@Component
@Lazy
public class ReportService {

    public ReportService() {
        System.out.println("ReportService created");
    }
}
```

```
package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        System.out.println("Application started");
    }
}
```

Question

Will `ReportService` be created during container startup?

What will be the output?

Question 3: Lazy Bean Requested Manu

Code

```
package in.strikes;

import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Component;

@Component
@Lazy
public class ReportService {

    public ReportService() {
        System.out.println("ReportService created");
    }

    public void generateReport() {
        System.out.println("Report generated");
    }
}
```

```
package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {
```

```

        ApplicationContext context =
            new AnnotationConfigApplicatio
fig.class);

        System.out.println("Application starte

        ReportService reportService = context.
ervice.class);
        reportService.generateReport();
    }
}

```

Question

What will be the output?

Question 4: Lazy Bean Needed by Eager Singleton

Code

```

package in.strikes;

import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Component;

@Component
@Lazy
public class ReportService {

    public ReportService() {
        System.out.println("ReportService created");
    }
}

```

```

package in.strikes;

import org.springframework.stereotype.Component

@Component
public class DashboardService {

    public DashboardService(ReportService reportService) {
        System.out.println("DashboardService created");
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppCon
fig.class);

        System.out.println("Application started");
    }
}

```

Question

`ReportService` is marked `@Lazy`.

Will it still be created during startup?

What will be the output?

Question 5: @Lazy on Injection Point

Code

```
package in.strikes;

import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Component;

@Component
@Lazy
public class EmailService {

    public EmailService() {
        System.out.println("EmailService created");
    }

    public void sendEmail() {
        System.out.println("Email sent");
    }
}
```

```
package in.strikes;

import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Component;

@Component
public class UserService {

    private final EmailService emailService;

    public UserService(@Lazy EmailService emailService) {
        this.emailService = emailService;
    }
}
```

```

        System.out.println("UserService create
    }

    public void registerUser() {
        System.out.println("User registered");
        emailService.sendEmail();
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        System.out.println("Application started");

        UserService userService = context.getBean(UserService.class);

        System.out.println("Before registerUser()");
        userService.registerUser();
    }
}

```

Question

What will be the output?

Question 6: @Lazy Only on Bean, Not on Point

Code

```
package in.strikes;

import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Component;

@Component
@Lazy
public class EmailService {

    public EmailService() {
        System.out.println("EmailService created");
    }

    public void sendEmail() {
        System.out.println("Email sent");
    }

}
```

```
package in.strikes;

import org.springframework.stereotype.Component;

@Component
public class UserService {

    private final EmailService emailService;

    public UserService(EmailService emailService) {
        this.emailService = emailService;
        System.out.println("UserService created");
    }

}
```

```

    }

    public void registerUser() {
        System.out.println("User registered");
        emailService.sendEmail();
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfi
fig.class);

        System.out.println("Application started");

        UserService userService = context.getBean(UserService.class);

        System.out.println("Before registerUser()");
        userService.registerUser();
    }
}

```

Question

`EmailService` is marked `@Lazy`.

Will it still be created during startup?

What will be the output?

Question 7: Prototype Bean Requested T

Code

```
package in.strikes;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("prototype")
public class OrderRequest {

    public OrderRequest() {
        System.out.println("OrderRequest created");
    }
}
```

```
package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppCon
fig.class);

        System.out.println("Application started");
    }
}
```

```

        OrderRequest r1 = context.getBean(Order
s);
        OrderRequest r2 = context.getBean(Order
s);

        System.out.println(r1 == r2);
    }
}

```

Question

What will be the output?

Question 8: Prototype Bean Injected Into Singleton Bean

Code

```

package in.strikes;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("prototype")
public class OrderRequest {

    private static int counter = 0;
    private final int id = ++counter;

    public OrderRequest() {
        System.out.println("OrderRequest created: " + id);
    }
}

```

```

        public int getId() {
            return id;
        }
    }
}

```

```

package in.strikes;

import org.springframework.stereotype.Component;

@Component
public class OrderService {

    private final OrderRequest orderRequest;

    public OrderService(OrderRequest orderRequest) {
        this.orderRequest = orderRequest;
        System.out.println("OrderService created");
    }

    public void placeOrder() {
        System.out.println("Using OrderRequest: " + orderRequest.getId());
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

```

```

        ApplicationContext context =
            new AnnotationConfigApplicatio
fig.class);

        System.out.println("Application starte

        OrderService orderService = context.ge
ice.class);

        orderService.placeOrder();
        orderService.placeOrder();
    }
}

```

Question

Will `OrderService` get a new `OrderRequest` every time `placeOrder()` is called?

What will be the output?

Question 9: Getting Fresh Prototype Bean Inside Singleton

Code

```

package in.strikes;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("prototype")
public class OrderRequest {

    private static int counter = 0;
    private final int id = ++counter;
}

```

```

    public OrderRequest() {
        System.out.println("OrderRequest creat
    }

    public int getId() {
        return id;
    }
}

```

```

package in.strikes;

import org.springframework.beans.factory.ObjectProvider;
import org.springframework.stereotype.Component;

@Component
public class OrderService {

    private final ObjectProvider<OrderRequest> orderRequestPr
    ovider;

    public OrderService(ObjectProvider<OrderRequest> orderReq
    uestProvider) {
        this.orderRequestProvider = orderRequestProvider;
        System.out.println("OrderService created");
    }

    public void placeOrder() {
        OrderRequest orderRequest = orderRequestProvider.getO
        bject();
        System.out.println("Using OrderRequest: " + orderRequ
        est.getId());
    }
}

```

```

package in.strikes;

import org.springframework.context.Application
import org.springframework.context.annotation.
gApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppCon
fig.class);

        System.out.println("Application started");

        OrderService orderService = context.getBean(OrderServ
ice.class);

        orderService.placeOrder();
        orderService.placeOrder();
    }
}

```

Question

What will be the output?

Question 10: Constructor-Based Circular Dependency

Code

```

package in.strikes;

import org.springframework.stereotype.Component;

```

```

@Component
public class OrderService {

    public OrderService(PaymentService payment) {
        System.out.println("OrderService created");
    }
}

```

```

package in.strikes;

import org.springframework.stereotype.Component;

@Component
public class PaymentService {

    public PaymentService(OrderService orderService) {
        System.out.println("PaymentService created");
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        System.out.println("Application started");
    }
}

```

```
}  
}
```

Question

Will the application start?

What will be the output?

Question 11: Circular Dependency Solved Using @Lazy

Code

```
package in.strikes;  
  
import org.springframework.context.annotation.Lazy;  
import org.springframework.stereotype.Component;  
  
@Component  
public class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(@Lazy PaymentService paymentService)  
    {  
        this.paymentService = paymentService;  
        System.out.println("OrderService created");  
    }  
  
    public void placeOrder() {  
        System.out.println("Order placed");  
        paymentService.pay();  
    }  
}
```

```

package in.strikes;

import org.springframework.context.annotation.
import org.springframework.stereotype.Component

@Component
@Lazy
public class PaymentService {

    private final OrderService orderService;

    public PaymentService(OrderService orderService) {
        this.orderService = orderService;
        System.out.println("PaymentService created");
    }

    public void pay() {
        System.out.println("Payment done");
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppCon
fig.class);
    }
}

```

```

        System.out.println("Application starte

        OrderService orderService = context.ge
ice.class);
        orderService.placeOrder();
    }
}

```

Question

Will this application start?

What will be the output?

Question 12: Setter-Based Circular Dependency in Spring Core

Code

```

package in.strikes;

import org.springframework.beans.factory.annotation.Autowired
d;
import org.springframework.stereotype.Component;

@Component
public class OrderService {

    private PaymentService paymentService;

    public OrderService() {
        System.out.println("OrderService constructor");
    }

    @Autowired
    public void setPaymentService(PaymentService paymentServi

```

```

ce) {
    this.paymentService = paymentService;
    System.out.println("PaymentService injected");
}
}

```

```

package in.strikes;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class PaymentService {

    private OrderService orderService;

    public PaymentService() {
        System.out.println("PaymentService constructor");
    }

    @Autowired
    public void setOrderService(OrderService orderService) {
        this.orderService = orderService;
        System.out.println("OrderService injected into PaymentService");
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfig

```

```

gApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicatio
fig.class);

        System.out.println("Application started");
    }
}

```

Question

Will this application start in Spring Core?

What will be the output?

Question 13: Field-Based Circular Dependency in Spring Core

Code

```

package in.strikes;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}

```

```

    public OrderService() {
        System.out.println("OrderService creat
    }
}

```

```

package in.strikes;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class PaymentService {

    @Autowired
    private OrderService orderService;

    public PaymentService() {
        System.out.println("PaymentService created");
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppCon
fig.class);
    }
}

```

```
        System.out.println("Application starte
    }
}
```

Question

Will this application start in Spring Core?

What will be the output?

Question 14: `@Bean` Singleton Creation

Code

```
package in.strikes;

public class User {

    public User() {
        System.out.println("User object created");
    }
}
```

```
package in.strikes;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public User user1() {
        return new User();
    }
}
```

```

    }

    @Bean
    public User user2() {
        return new User();
    }
}

```

```

package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfi
fig.class);

        System.out.println("Application started");

        User u1 = context.getBean("user1", User.class);
        User u2 = context.getBean("user1", User.class);
        User u3 = context.getBean("user2", User.class);

        System.out.println(u1 == u2);
        System.out.println(u1 == u3);
    }
}

```

Question

How many `User` objects will Spring create?

What will be the output?

Final Revision Questions

Revision Question A

If a singleton bean is marked `@Lazy`, will Spring create it during startup?

Revision Question B

If a prototype bean is injected into a singleton bean, will the singleton get a new prototype object every time its method runs?

Revision Question C

What is the difference between `@Lazy` on a class and `@Lazy` on a constructor parameter?

Revision Question D

Why does constructor-based circular dependency fail?

Revision Question E

Why can setter or field circular dependency sometimes work in Spring Core?

Revision Question F

What is the best solution for circular dependency?

Answer Key With Explanations

Answer 1: Basic Eager Initialization

Output

```
PaymentGateway created
PaymentService created
OrderService created
Application started
```

Explanation

All three beans are singleton beans.

Singleton beans are eagerly created when the Spring container starts.

Spring creates dependencies first:

```
PaymentGateway
  ↓
PaymentService
  ↓
OrderService
```

So `PaymentGateway` is created before `PaymentService`, and `PaymentService` is created before `OrderService`.

Answer 2: Lazy Bean Not Requested

Output

```
Application started
```

Explanation

`ReportService` is marked with `@Lazy`.

So Spring knows about this bean, but it does not create the actual object during startup.

Spring only stores its bean definition.

The object will be created only when someone asks for it.

Answer 3: Lazy Bean Requested Manual

Output

```
Application started
ReportService created
Report generated
```

Explanation

`ReportService` is lazy, so it is not created when the container starts.

It is created only when this line runs:

```
ReportService reportService = context.getBean(ReportService.class);
```

After that, the method runs:

```
reportService.generateReport();
```

Answer 4: Lazy Bean Needed by Eager Singleton

Output

```
ReportService created
DashboardService created
Application started
```

Explanation

`DashboardService` is a normal singleton bean.

Normal singleton beans are eagerly created during container startup.

But `DashboardService` needs `ReportService` in its constructor.

So Spring must create `ReportService` also, even though it is m
Important point:

A lazy bean may still be created during startup if an eager needs it.

Answer 5: `@Lazy` on Injection Point

Output

```
UserService created
Application started
Before registerUser()
User registered
EmailService created
Email sent
```

Explanation

`EmailService` is marked `@Lazy`.

Also, the injection point inside `UserService` is marked with `@Lazy`.

```
public UserService(@Lazy EmailService emailService)
```

So Spring does not inject the real `EmailService` immediately.

It injects a proxy object.

The real `EmailService` object is created only when this line runs:

```
emailService.sendEmail();
```

That is why `EmailService created` is printed only after `User registered`.

Answer 6: `@Lazy` Only on Bean, Not on Injection Point

Output

```
EmailService created
UserService created
Application started
Before registerUser()
User registered
Email sent
```

Explanation

`UserService` is an eager singleton bean.

To create `UserService`, Spring needs `EmailService`.

Since the injection point is not marked `@Lazy`, Spring has to provide the real `EmailService` object immediately.

So `EmailService` is created during startup.

Important difference:

```
@Lazy on bean:
Create this bean only when needed.

@Lazy on injection point:
Inject proxy and resolve real bean later.
```

Answer 7: Prototype Bean Requested Twice

Output

```
Application started
OrderRequest created
OrderRequest created
false
```

Explanation

Prototype beans are not created during startup.

They are created when requested from the container.

Here, we request the bean twice:

```
context.getBean(OrderRequest.class);  
context.getBean(OrderRequest.class);
```

So Spring creates two different objects.

That is why:

```
r1 == r2
```

prints:

```
false
```

Answer 8: Prototype Bean Injected Into Singleton Bean

Output

```
OrderRequest created: 1  
OrderService created  
Application started  
Using OrderRequest: 1  
Using OrderRequest: 1
```

Explanation

`OrderService` is singleton.

So Spring creates `OrderService` only once during startup.

At the time of creating `OrderService`, Spring injects one `OrderRequest` object.

After that, the same `OrderRequest` object remains inside `OrderService`.

Prototype means:

New object whenever the prototype bean is requested from the container.

It does not mean:

New object every time a singleton method runs.

Answer 9: Getting Fresh Prototype Bean Inside Singleton

Output

```
OrderService created
Application started
OrderRequest created: 1
Using OrderRequest: 1
OrderRequest created: 2
Using OrderRequest: 2
```

Explanation

This time `OrderService` does not directly store one `OrderRequest`.

It stores:

```
ObjectProvider<OrderRequest>
```

Every time this line runs:

```
orderRequestProvider.getObject();
```

Spring is asked for a fresh prototype bean.

So a new `OrderRequest` object is created for every call.

Answer 10: Constructor-Based Circular Dependency

Output

The application will fail during container startup.

You will get an exception related to circular dependency.

Commonly, Spring throws something like:

```
BeanCurrentlyInCreationException
```

`Application started` will not be printed.

Explanation

To create `OrderService`, Spring needs `PaymentService`.

To create `PaymentService`, Spring needs `OrderService`.

With constructor injection, Spring cannot create either object first.

Constructor injection requires all dependencies before object creation.

So there is no valid starting point.

Answer 11: Circular Dependency Solved Using `@Lazy`

Output

```
OrderService created
Application started
Order placed
PaymentService created
Payment done
```

Explanation

Here, `OrderService` depends on `PaymentService`.

But the constructor parameter is marked with `@Lazy`:

```
public OrderService(@Lazy PaymentService paymentService)
```

So Spring injects a proxy instead of the real `PaymentService`.

Also, `PaymentService` itself is marked with `@Lazy`, so it is not cre

The real `PaymentService` is created only when this line runs:

```
paymentService.pay();
```

This can solve the circular dependency technically.

But it should be treated as a workaround, not the best design.

The better solution is to refactor the code and remove the circular dependency.

Answer 12: Setter-Based Circular Dependency in Spring Core

Output

The application may start successfully.

A possible output is:

```
OrderService constructor  
PaymentService constructor  
OrderService injected into PaymentService  
PaymentService injected into OrderService  
Application started
```

Explanation

This is setter injection.

In setter injection, object creation and dependency injection are separate steps.

Spring can first create the objects using no-argument constructors:

```
Create OrderService object  
Create PaymentService object
```

Then Spring can inject dependencies later:

```
Inject PaymentService into OrderService  
Inject OrderService into PaymentService
```

That is why setter-based circular dependency can sometime Core.

But this is still a design smell.

A working circular dependency is not automatically a good design.

Answer 13: Field-Based Circular Dependency in Spring Core

Output

The application may start successfully.

A possible output is:

```
OrderService created  
PaymentService created  
Application started
```

Explanation

Field injection separates object creation from dependency injection.

Spring can create an early reference of one bean and use it to complete the other bean.

This is why some setter/field-based circular dependencies may work in Spring Core.

But this does not mean field injection is recommended.

The better practice is to remove the circular dependency by improving the design.

Answer 14: @Bean Singleton Creation

Output

```
User object created
User object created
Application started
true
false
```

Explanation

There are two bean definitions:

```
user1
user2
```

Both are singleton by default.

So Spring creates one object for `user1` and one object for `user2`.

That means:

```
u1 == u2
```

prints:

```
true
```

because both are coming from the same bean definition.

But:

```
u1 == u3
```

prints:

```
false
```

because `user1` and `user2` are two different bean definitions.

Important point:

Spring singleton means one object per bean definition inside one object per class.

Revision Answers

Revision Answer A

If a singleton bean is marked `@Lazy`, Spring will not create it during startup by default.

Spring will create it only when it is requested.

But if an eager singleton directly depends on it, Spring may create it during startup to satisfy that dependency.

Revision Answer B

No.

If a prototype bean is injected into a singleton bean, the prototype bean is created once at the time of singleton bean creation.

The same prototype reference is reused inside the singleton.

Revision Answer C

`@Lazy` on a class means:

Do not create this bean until it is needed.

`@Lazy` on a constructor parameter means:

Inject a proxy first.
Resolve the real dependency later when it is used.

Revision Answer D

Constructor-based circular dependency fails because constructors create all dependencies before object creation.

If **A** needs **B** in its constructor and **B** needs **A** in its constructor, you cannot create either one first.

There is no starting point.

Revision Answer E

Setter or field circular dependency can sometimes work in Spring Core because object creation and dependency injection are separate steps.

Spring can create the object first and inject dependencies later using an early reference.

But this should not be treated as a good design.

Revision Answer F

The best solution for circular dependency is refactoring.

Possible refactoring options:

1. Move common logic to a third service.
2. Split responsibilities clearly.
3. Introduce an orchestrator class.
4. Use events where suitable.
5. Redesign the dependency direction.

Circular dependency usually means two classes know too much about each other.